Software Design Document

1. Introduction

This is the Phase 3 software design document for the Moderated Learning Management System (C++17, SQLite, Waterfall). It turns the Phase 2 SRS (FR01–FR19, NFR1–NFR9, BR01 and BR2–BR6) and the use case model (UC01–UC24) into a design that can be coded without further decisions.

The design is driven by five requirements:

- **NFR3 and NFR9:** the system runs offline and keeps all data in one SQLite file on the host computer.
- **NFR4:** passwords are stored as Argon2id hashes (libsodium).
- **NFR6 and NFR7:** a crash never leaves a partly saved submission or mark, and a mark change and its audit entry are saved in one transaction.
- **NFR1 and NFR2:** every operation finishes within 2 seconds with 800 students.
- **NFR8:** at least 70% line coverage of the business-logic layer, so the business logic must be testable without the console or the database.

The same class, table and state names are used in every diagram, the SQL and the traceability table in section 6.

Section 2 gives the architecture, section 3 the class diagram, section 4 the behavioural models, section 5 the database design, section 6 the traceability matrix and section 7 the decisions to confirm at Gate 2.

2. Architecture

2.1 Architectural style: layered architecture

The system is a four-layer architecture. Each layer may only call the layer directly below it, and the lower layers never call upward.

- **Presentation:** the console UI. It reads input, calls an application service and prints the result. It holds no business rules.
- **Application services:** AuthService, AdminService, AssignmentService, SubmissionService, MarkingService and ModerationService. Each implements one group of use cases and owns the transaction boundary.
- **Domain:** entities, the status rules, the late-penalty policy and UserFactory. This layer has no dependency on SQLite or on the console.
- **Infrastructure:** the SQLite repositories, database access and transaction, the password hasher (libsodium), and the system clock.

Why this style fits:

1. The business rules (BR01, BR2–BR6) live in one layer that can be unit tested with fake repositories and a fake clock. That is how NFR8 (70% coverage) is met.
2. The UI and the database can change without touching the rules (for example, a GUI later).
3. The services own the transaction, so a mark change plus its audit entry is saved atomically (NFR7) and a submission is saved whole or not at all (NFR6).

4. Dependencies are inverted at the domain boundary: the services use repository interfaces, and SQLite classes implement them (see the Repository pattern in section 3).

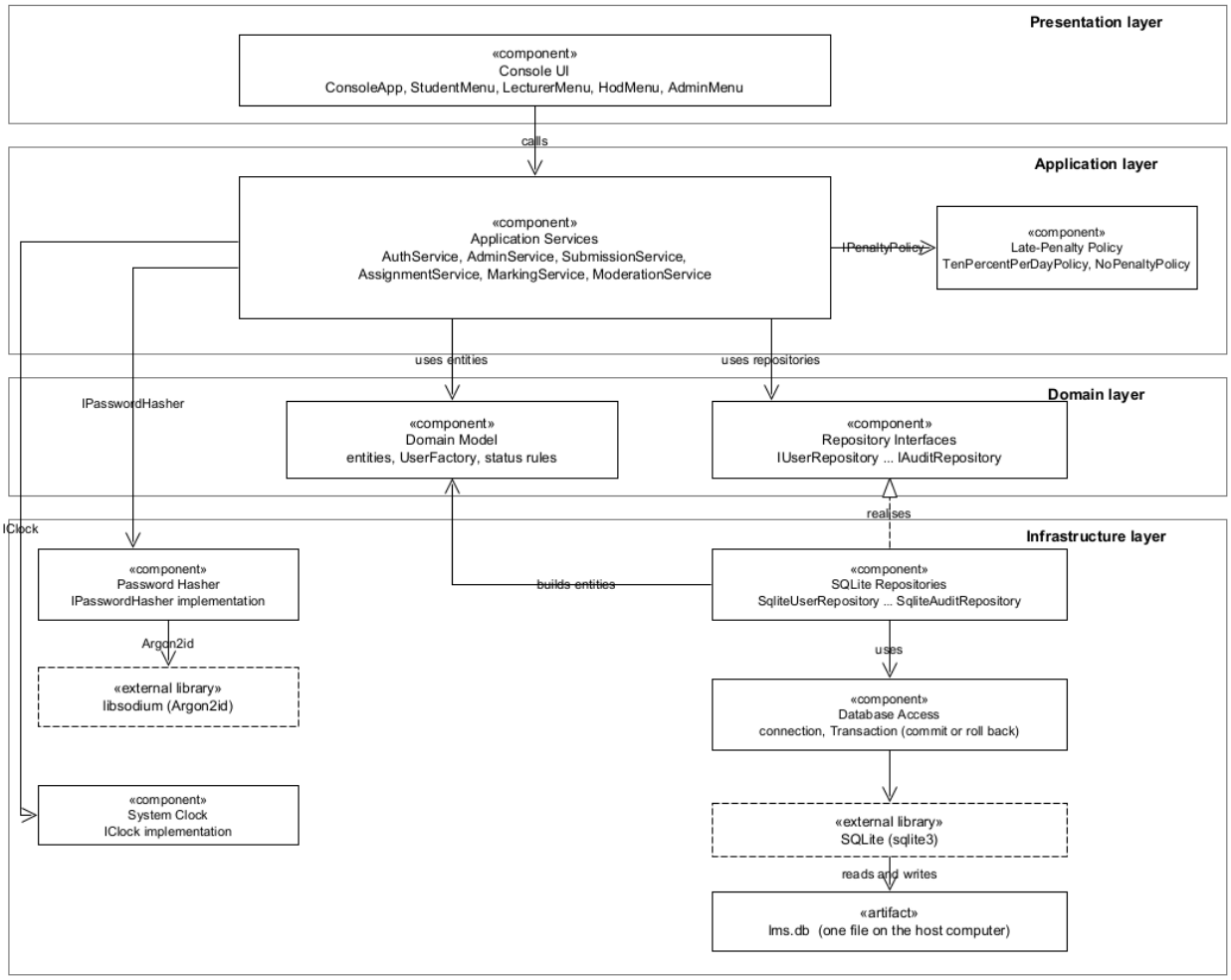


Figure 1. Component diagram. Each box is a component, grouped by layer. Arrows point from the user of an interface to the provider.

2.2 Components

Component	Responsibility	Main requirements
Console UI	Menus, input, output	All FRs (entry point)
Application Services	One service per use case group; transactions; role checks	FR01–FR19
Late-Penalty Policy	Calculates the penalty (BR01)	FR12, BR01
Domain Model	Entities, status transitions, UserFactory	BR2–BR6
Repository Interfaces	What the services need from storage	NFR8
SQLite Repositories	Implement the interfaces; build entities from rows	NFR9
Database Access	Connection and Transaction	NFR6, NFR7
Password Hasher	Argon2id hashing and checking	NFR4
System Clock	Current time (replaceable in tests)	BR01, deadlines
lms.db	The one SQLite file on the host	NFR3, NFR9

2.3 Deployment assumption

The application and lms.db run on one host computer. Users reach it at the host, or by remote terminal (for example SSH) to that computer. There is no network protocol of our own and no second server. This keeps NFR3 (offline) and NFR9 (one file) true.

2.4 Design rationale and rejected alternatives

Rejected alternative 1: MVC. MVC suits an application with many interactive views over a shared model. Here the user interface is a console with simple menus, so a separate Controller adds a class per screen and no benefit. Business rules also tend to end up inside controllers, which makes them hard to test and puts NFR8 at risk. The layered style keeps the rules in services and the domain, and the console is only a thin presentation layer.

Rejected alternative 2: client-server over a network. A separate server process with clients would need a network protocol, authentication of sessions, and a database server or shared file access. This conflicts with NFR3 (offline) and NFR9 (all data in one SQLite file), adds failure modes that Waterfall testing would have to cover, and is more than 800 students and one host need. SQLite, which allows one writer at a time, is sufficient at this size.

Accepted trade-off. A layered design adds interfaces and some mapping code between rows and entities. We accept this cost because it buys testability (NFR8) and keeps the rules in one place.

3. Class diagram

The class diagram is design-level: it shows visibility (+ public, - private), types, multiplicities, inheritance and interfaces. It is one model drawn in three figures so that each stays readable. Figure 2 is the domain model, Figure 3 the application services, and Figure 4 the interfaces (ports), the late-penalty strategy and the infrastructure classes that implement them.

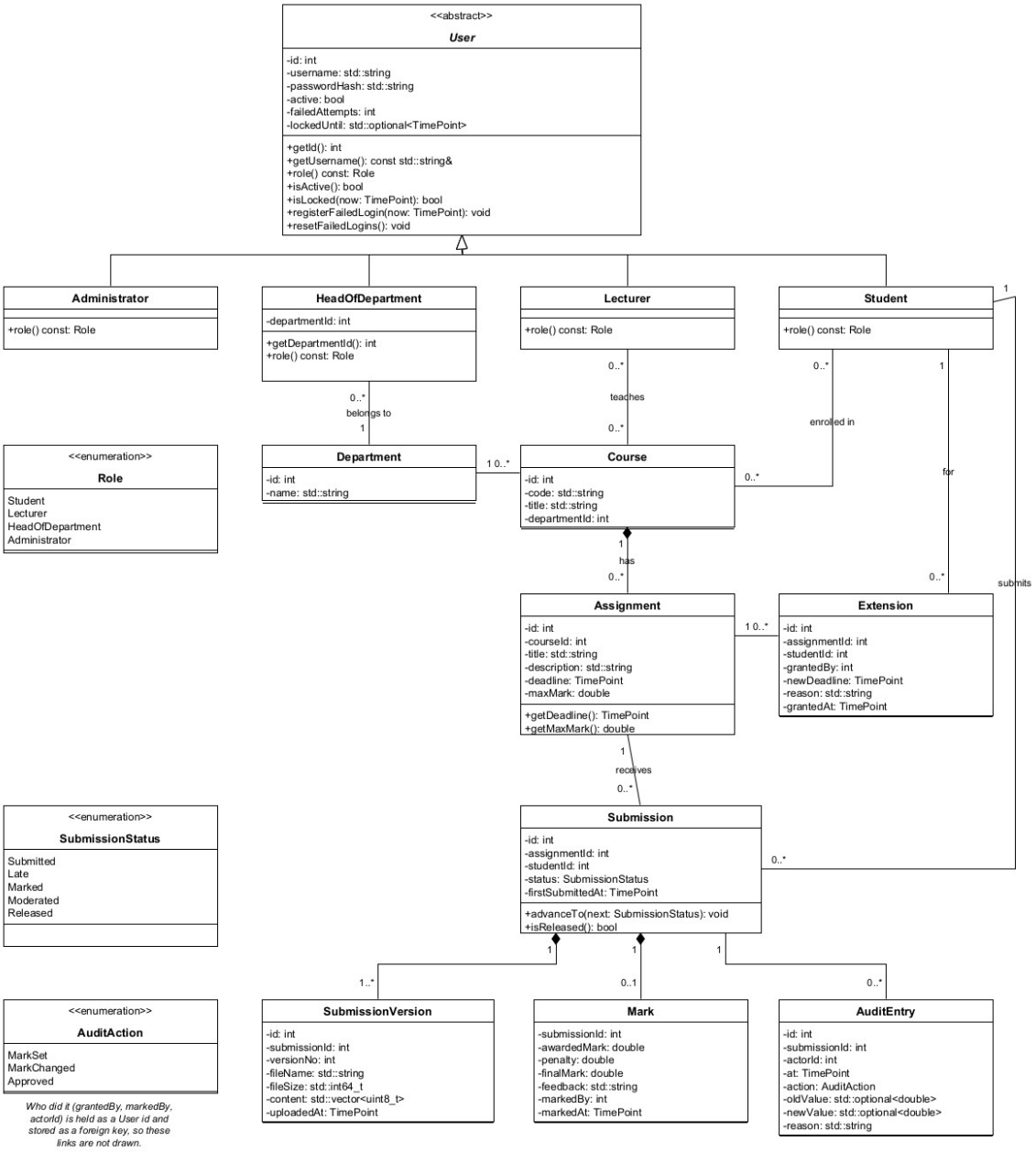


Figure 2. Class diagram A: domain model. User is abstract with four subclasses (inheritance). Submission is composed of its versions and its mark.

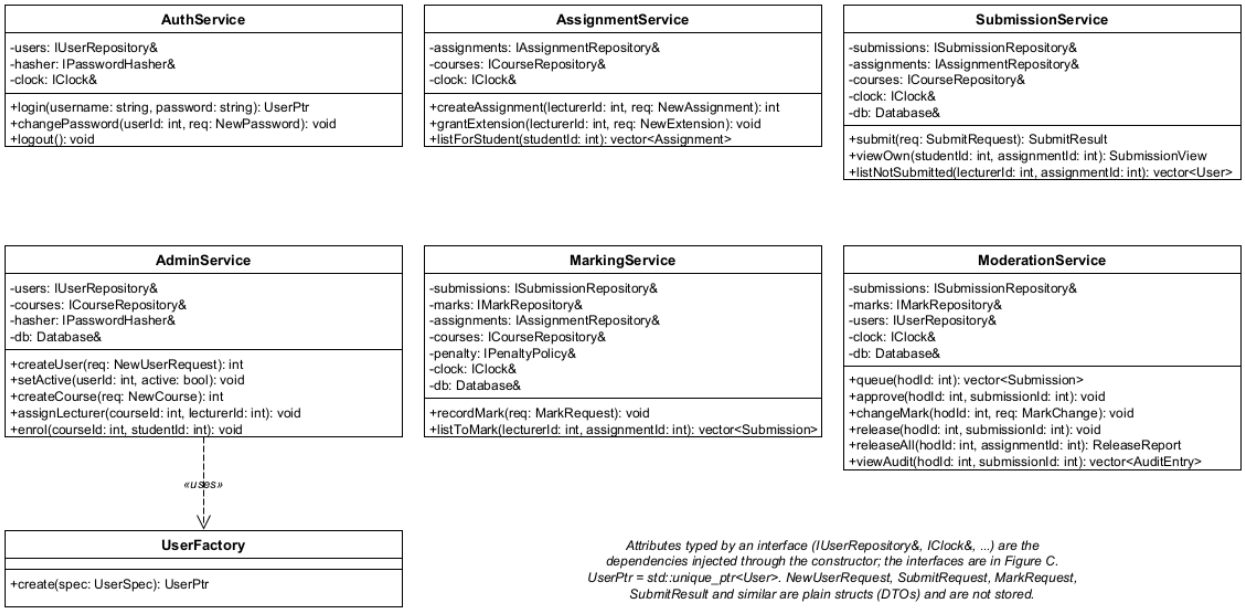


Figure 3. Class diagram B: application services and UserFactory. Interface-typed attributes are the injected dependencies.

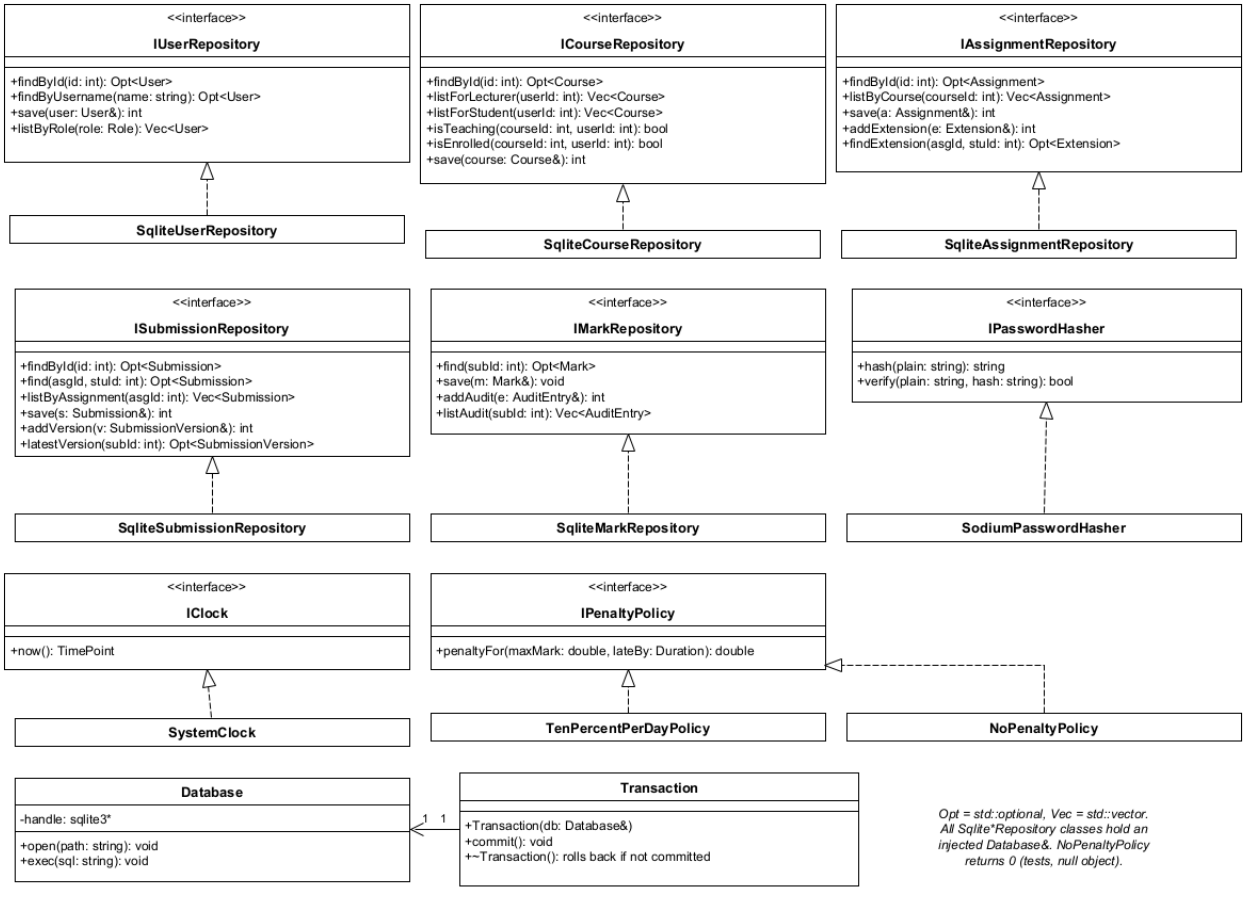


Figure 4. Class diagram C: interfaces, the penalty strategy and their implementations. Dashed arrows with a hollow triangle mean "implements".

3.1 Design patterns

Pattern	Where	Why it was chosen
Strategy	IPenaltyPolicy with TenPercentPerDayPolicy and NoPenaltyPolicy	BR01 is a rule that a department may change (the percentage, the number of days). MarkingService only calls penaltyFor(maxMark, lateBy), so a different rule is a new class and no existing code changes. TenPercentPerDayPolicy returns 10% of the maximum mark for each started 24 hours late, up to 3 days, and the full maximum mark after that, so the final mark becomes 0. NoPenaltyPolicy returns 0; it exists mainly as a test double and null object, so tests of marking can run without the late rule.
Repository	IUserRepository, ICourseRepository, IAssignmentRepository, ISubmissionRepository, IMarkRepository and their SQLite implementations	Services ask for entities and never write SQL. This hides SQLite behind interfaces, so unit tests use in-memory fakes (NFR8) and the table layout can change without touching business rules.
Factory (simple factory)	UserFactory.create(spec)	Creating a user needs one of four subclasses chosen by role, and HeadOfDepartment needs a department. UserFactory is the only place that knows this, both when an Administrator creates an account and when a repository rebuilds a User from a row.

Not used on purpose. The submission life cycle has five states, so we use an enum (SubmissionStatus) and one guarded method, Submission.advanceTo(next), which rejects any transition not in the state diagram. The State pattern would add five classes for the same behaviour.

3.2 How the design follows SOLID

Principle	How the design follows it
Single responsibility	One service per use case group (Auth, Admin, Assignment, Submission, Marking, Moderation). The console only talks to the user, repositories only store, the policy only calculates a penalty.
Open/closed	A new penalty rule is a new IPenaltyPolicy class. A new storage engine is a new set of repository classes. No service changes.
Liskov substitution	Administrator, HeadOfDepartment, Lecturer and Student can all be used where a User is expected. NoPenaltyPolicy and TenPercentPerDayPolicy keep the same contract (a penalty from 0 to the maximum mark), and fake repositories in tests keep the contract of the real ones.
Interface segregation	Five small repository interfaces and separate IClock, IPasswordHasher and IPenaltyPolicy, instead of one large database interface. AuthService depends only on IUserRepository, IPasswordHasher and IClock.
Dependency inversion	Services depend on interfaces, never on Sqlite* classes or libsodium. The composition root (main) creates the concrete objects and passes them to the services through constructors.

4. Behavioural models

4.1 State machine: Submission life cycle

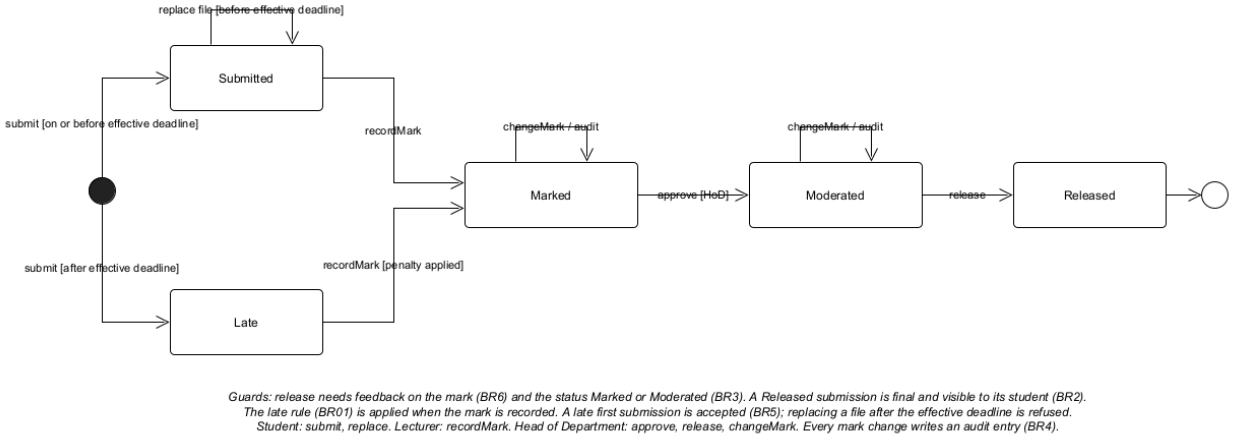


Figure 5. State machine for Submission.

A submission starts as **Submitted** (first file on or before the effective deadline) or **Late** (first file after it). A Submitted submission can be replaced by a new file version until the effective deadline. A Lecturer's recordMark moves it to **Marked** (the late penalty is applied here). The Head of Department may change the mark in Marked or Moderated, and each change writes an audit entry. approve moves Marked to **Moderated**, and release moves it to **Released**. Releasing a Marked submission counts as approval, so it passes through Moderated and writes the approval entry in one action. Release needs feedback (BR6) and a moderated or marked status (BR3). Only Released marks are visible to the student (BR2).

4.2 Activity diagram with swimlanes: submission to released mark

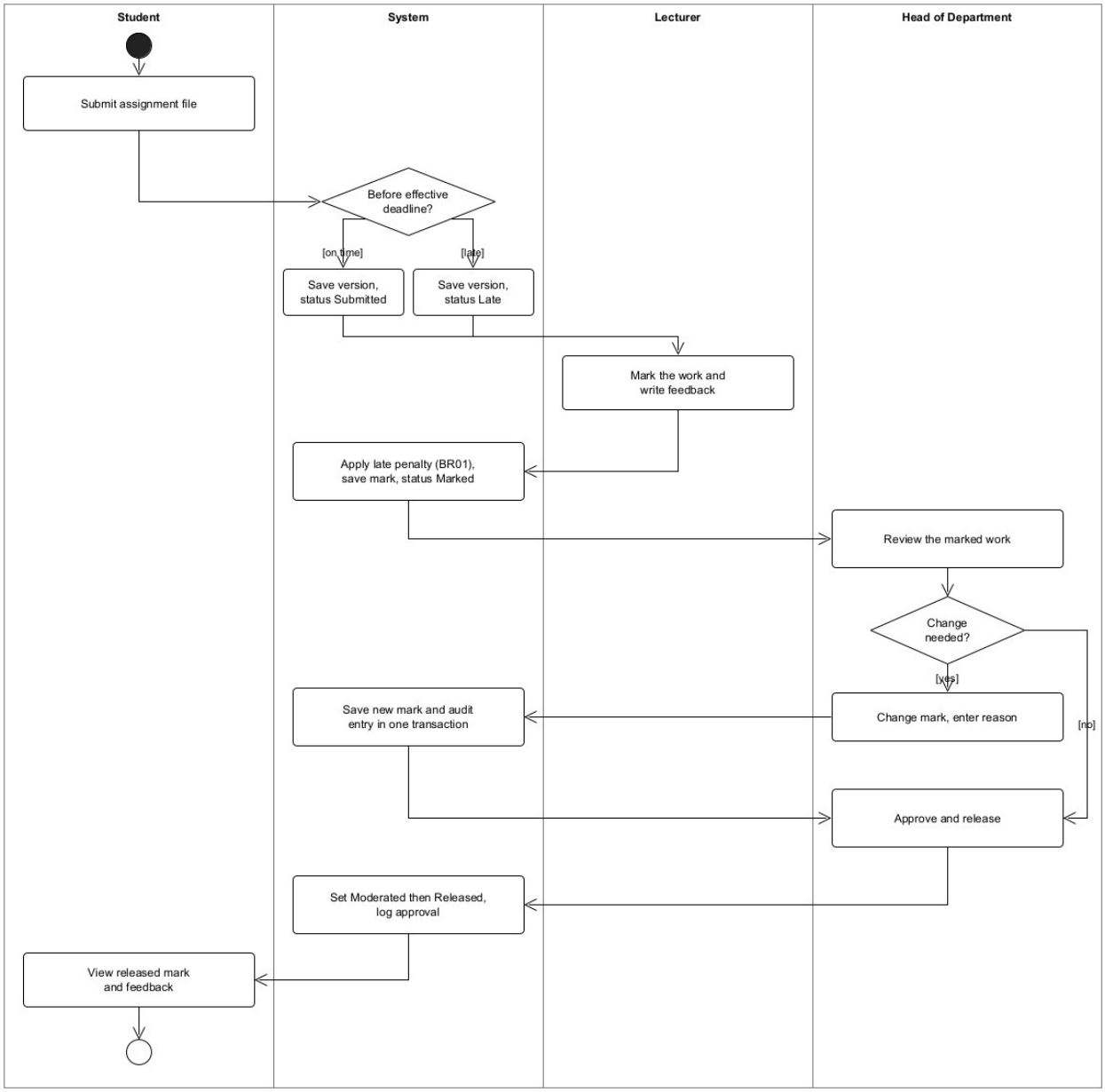


Figure 6. Activity diagram. Lanes: Student, System, Lecturer and Head of Department.

4.3 Sequence diagram: Submit assignment

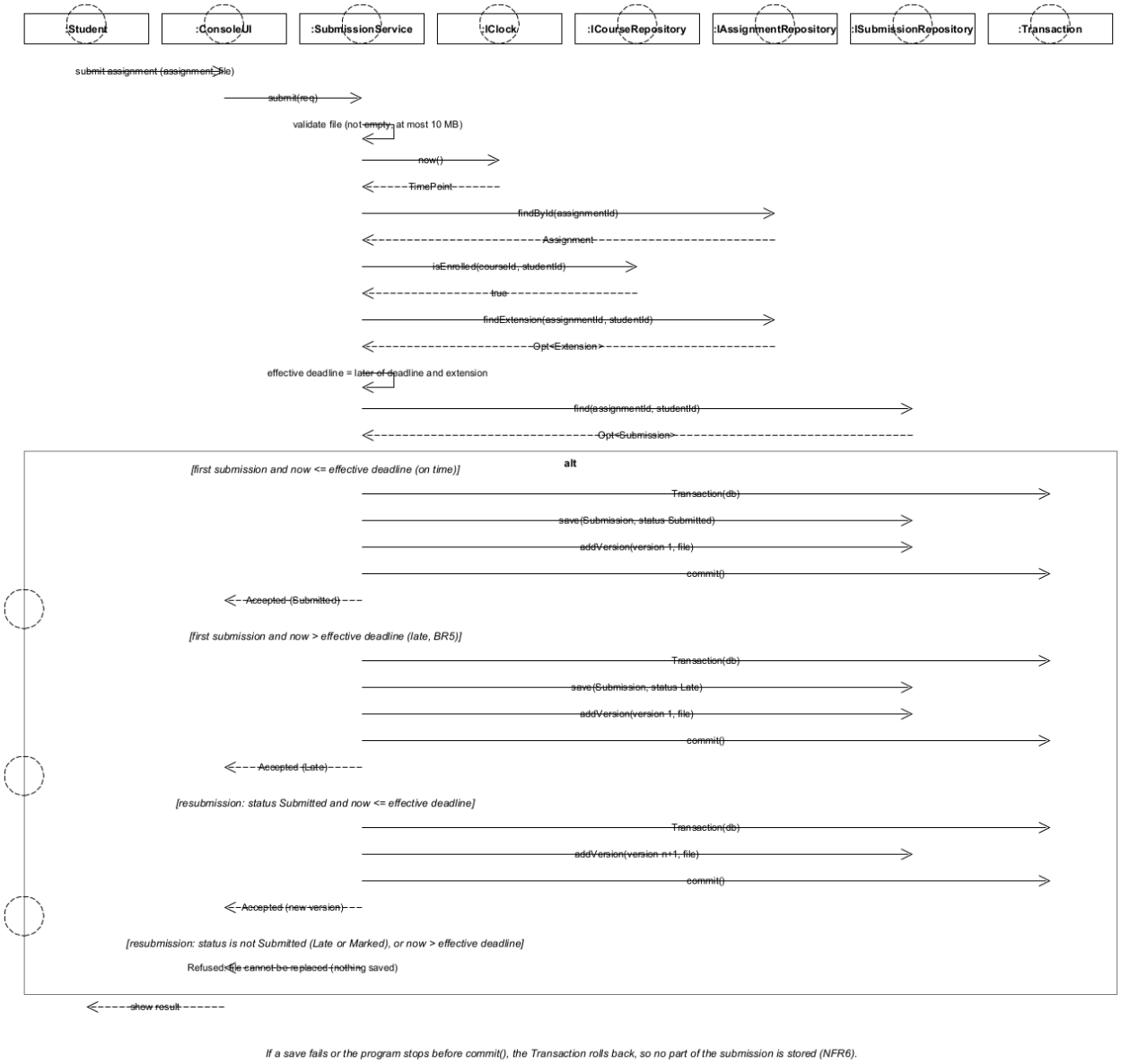


Figure 7. Submit assignment. The alt fragment has four paths: first on time, first late, resubmission before the deadline, and a refused resubmission.

The effective deadline is the later of the assignment deadline and any extension granted to that student (UC07 and UC13). A first submission after it is accepted as Late, never rejected (BR5). Replacing a file is allowed only while the status is Submitted and the effective deadline has not passed. Every path that writes uses one Transaction, so a failure leaves nothing saved (NFR6).

4.4 Sequence diagram: Moderate and release marks

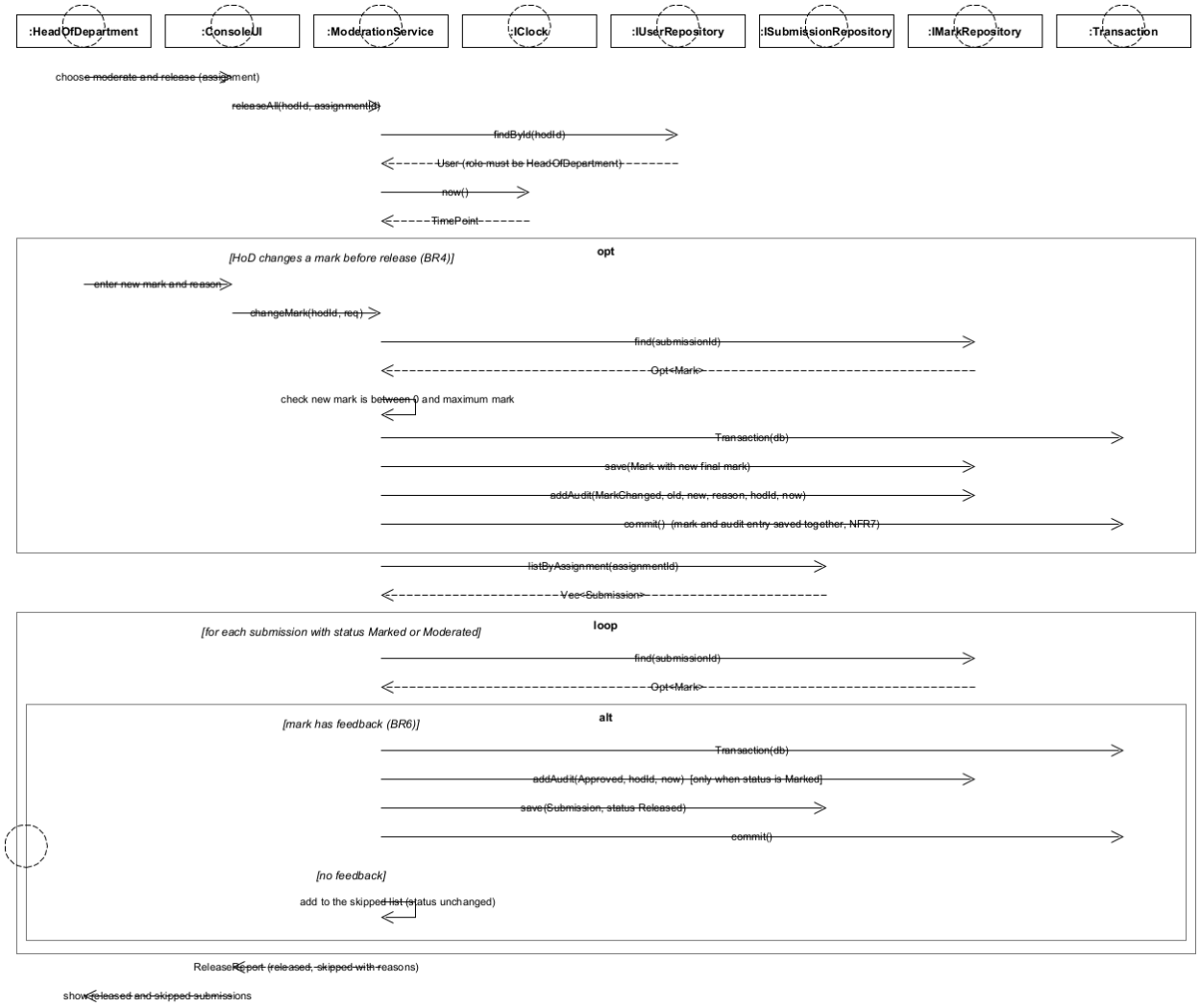


Figure 8. Moderate and release marks. The opt fragment is the optional mark change; the loop releases each submission.

A mark change and its audit entry are saved in one transaction (NFR7). In bulk release, a submission without feedback is skipped and listed in the ReleaseReport, so the Head of Department sees exactly what was not released and why.

5. Database design

All data is in one SQLite file, lms.db (NFR9). The schema has 11 tables. It was written as SQL, run in SQLite and tested: every CHECK, UNIQUE and foreign key below was exercised with a bad insert and rejected, and the two triggers were tested too.

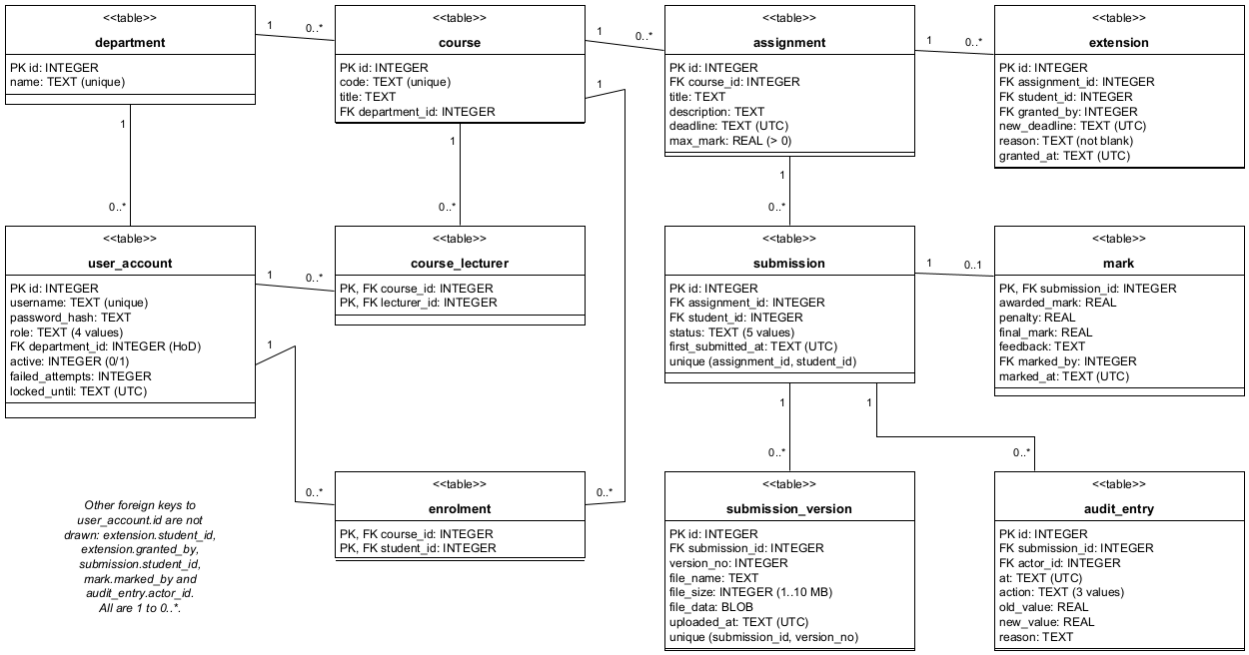


Figure 9. ERD. PK = primary key, FK = foreign key. Labels give the multiplicity at each end (1 and 0..). The other foreign keys to user_account are listed in the note on the figure.*

5.1 Normalisation to 3NF

- **1NF.** Every column holds one atomic value. Repeating data is in its own table: a student's several files are rows in submission_version, a course's several lecturers are rows in course_lecturer.
- **2NF.** Every table except course_lecturer and enrolment has a single-column primary key, so no column can depend on part of a key. The two composite-key tables hold only their two key columns, so they have nothing that could depend on part of the key.
- **3NF.** No non-key column depends on another non-key column. For example, course stores department_id but not the department name, submission stores assignment_id but not the deadline or maximum mark, and audit_entry stores actor_id but not the actor's name.

Two columns look like exceptions, and both are deliberate:

1. **mark.final_mark** is awarded_mark minus penalty (never below 0) at first marking, but the Head of Department can change the final mark directly (UC20), so it is a stored decision and not a calculation. Both the awarded mark and the final mark are kept, with the penalty, so the history is clear.
2. **submission.first_submitted_at** equals the uploaded_at of version 1. It is set once, in the same transaction that creates the submission, and kept so the late check and class lists need no join.

5.2 Data dictionary

Times are stored as ISO-8601 UTC text (for example 2026-11-01T12:00:00Z), which sorts correctly. Roles, statuses and audit actions are stored as the same words used by the enumerations in the class diagram.

department

Column	Type	Key	Rules and meaning
id	INTEGER	PK	Department number
name	TEXT		Not null, unique

user_account (all four roles)

Column	Type	Key	Rules and meaning
id	INTEGER	PK	
username	TEXT		Not null, unique
password_hash	TEXT		Not null. Argon2id string from libsodium (NFR4). The password itself is never stored
role	TEXT		Not null. One of Student, Lecturer, HeadOfDepartment, Administrator
department_id	INTEGER	FK department	Required for a HeadOfDepartment and null for every other role (table CHECK)
active	INTEGER		0 or 1, default 1. Set to 0 to disable an account
failed_attempts	INTEGER		0 or more, default 0. Counts wrong passwords in a row
locked_until	TEXT		Null, or the time the lock ends

course

Column	Type	Key	Rules and meaning
id	INTEGER	PK	
code	TEXT		Not null, unique (for example CS4433)
title	TEXT		Not null
department_id	INTEGER	FK department	Not null

course_lecturer (a course has many lecturers, a lecturer teaches many courses)

Column	Type	Key	Rules and meaning
course_id	INTEGER	PK, FK course	
lecturer_id	INTEGER	PK, FK user_account	Must be a Lecturer. A foreign key cannot check the role, so AdminService checks it

enrolment

Column	Type	Key	Rules and meaning
course_id	INTEGER	PK, FK course	
student_id	INTEGER	PK, FK user_account	Must be a Student, checked by AdminService

assignment

Column	Type	Key	Rules and meaning
id	INTEGER	PK	
course_id	INTEGER	FK course	Not null
title	TEXT		Not null
description	TEXT		Not null, default empty
deadline	TEXT		Not null. The deadline before any extension
max_mark	REAL		Not null, greater than 0

extension

Column	Type	Key	Rules and meaning
id	INTEGER	PK	
assignment_id	INTEGER	FK assignment	Not null
student_id	INTEGER	FK user_account	Not null. The student who gets the extension
granted_by	INTEGER	FK user_account	Not null. The lecturer

Column	Type	Key	Rules and meaning
new_deadline	TEXT		Not null
reason	TEXT		Not null, not blank
granted_at	TEXT		Not null. If a student has several extensions, the latest one counts

submission

Column	Type	Key	Rules and meaning
id	INTEGER	PK	
assignment_id	INTEGER	FK assignment	Not null
student_id	INTEGER	FK user_account	Not null
status	TEXT		Not null. One of Submitted, Late, Marked, Moderated, Released
first_submitted_at	TEXT		Not null
(table rule)		unique	(assignment_id, student_id): one submission per student per assignment

submission_version

Column	Type	Key	Rules and meaning
id	INTEGER	PK	
submission_id	INTEGER	FK submission	Not null
version_no	INTEGER		1 or more. Unique with submission_id. The highest number is the version that counts
file_name	TEXT		Not null
file_size	INTEGER		Bytes, from 1 to 10 485 760 (10 MB)
file_data	BLOB		Not null. The file itself, stored inside the database so that a submission is saved whole or not at all (NFR6, NFR9)
uploaded_at	TEXT		Not null

mark (at most one per submission)

Column	Type	Key	Rules and meaning
submission_id	INTEGER	PK, FK submission	One mark per submission
awarded_mark	REAL		0 or more. The lecturer's mark
penalty	REAL		0 or more, default 0. Set by the penalty policy (BR01)
final_mark	REAL		0 or more. What the student sees once Released
feedback	TEXT		Default empty. Must not be blank to release (BR6)
marked_by	INTEGER	FK user_account	Not null
marked_at	TEXT		Not null

audit_entry (append-only log, BR4)

Column	Type	Key	Rules and meaning
id	INTEGER	PK	
submission_id	INTEGER	FK submission	Not null
actor_id	INTEGER	FK user_account	Not null. Who did it
at	TEXT		Not null. When
action	TEXT		One of MarkSet, MarkChanged, Approved
old_value	REAL		Null when there was no earlier mark, or for Approved

Column	Type	Key	Rules and meaning
new_value	REAL		Null for Approved
reason	TEXT		Default empty. Must not be blank for MarkChanged

5.3 SQL schema

This is the exact script that was tested. Foreign keys must be switched on for each connection (PRAGMA foreign_keys = ON), and the Database class does this when it opens the file.

```
PRAGMA foreign_keys = ON;

CREATE TABLE department (
    id    INTEGER PRIMARY KEY,
    name  TEXT NOT NULL UNIQUE
);

CREATE TABLE user_account (
    id               INTEGER PRIMARY KEY,
    username         TEXT NOT NULL UNIQUE,
    password_hash    TEXT NOT NULL,
    role             TEXT NOT NULL
                     CHECK (role IN ('Student','Lecturer','HeadOfDepartment','Administrator')),
    department_id    INTEGER REFERENCES department(id),
    active           INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0,1)),
    failed_attempts  INTEGER NOT NULL DEFAULT 0 CHECK (failed_attempts >= 0),
    locked_until     TEXT,
    CHECK ((role = 'HeadOfDepartment' AND department_id IS NOT NULL)
        OR (role <> 'HeadOfDepartment' AND department_id IS NULL))
);

CREATE TABLE course (
    id             INTEGER PRIMARY KEY,
    code           TEXT NOT NULL UNIQUE,
    title          TEXT NOT NULL,
    department_id  INTEGER NOT NULL REFERENCES department(id)
);

CREATE TABLE course_lecturer (
    course_id    INTEGER NOT NULL REFERENCES course(id),
    lecturer_id  INTEGER NOT NULL REFERENCES user_account(id),
    PRIMARY KEY (course_id, lecturer_id)
);

CREATE TABLE enrolment (
    course_id   INTEGER NOT NULL REFERENCES course(id),
    student_id  INTEGER NOT NULL REFERENCES user_account(id),
    PRIMARY KEY (course_id, student_id)
);

CREATE TABLE assignment (
    id           INTEGER PRIMARY KEY,
    course_id    INTEGER NOT NULL REFERENCES course(id),
    title        TEXT NOT NULL,
    description  TEXT NOT NULL DEFAULT '',
    deadline     TEXT NOT NULL,
    max_mark     REAL NOT NULL CHECK (max_mark > 0)
);

CREATE TABLE extension (
    id            INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL REFERENCES assignment(id),
    student_id    INTEGER NOT NULL REFERENCES user_account(id),
    granted_by    INTEGER NOT NULL REFERENCES user_account(id),
    new_deadline  TEXT NOT NULL,
    reason        TEXT NOT NULL CHECK (length(trim(reason)) > 0),
    granted_at    TEXT NOT NULL
);

CREATE TABLE submission (
    id                  INTEGER PRIMARY KEY,
    assignment_id       INTEGER NOT NULL REFERENCES assignment(id),
    student_id          INTEGER NOT NULL REFERENCES user_account(id),
    status              TEXT NOT NULL
```

```
                        CHECK (status IN ('Submitted','Late','Marked','Moderated','Released')),
    first_submitted_at  TEXT NOT NULL,
    UNIQUE (assignment_id, student_id)
);

CREATE TABLE submission_version (
    id             INTEGER PRIMARY KEY,
    submission_id  INTEGER NOT NULL REFERENCES submission(id),
    version_no     INTEGER NOT NULL CHECK (version_no >= 1),
    file_name      TEXT NOT NULL,
    file_size      INTEGER NOT NULL CHECK (file_size BETWEEN 1 AND 10485760),
    file_data      BLOB NOT NULL,
    uploaded_at    TEXT NOT NULL,
    UNIQUE (submission_id, version_no)
);

CREATE TABLE mark (
    submission_id  INTEGER PRIMARY KEY REFERENCES submission(id),
    awarded_mark   REAL NOT NULL CHECK (awarded_mark >= 0),
    penalty        REAL NOT NULL DEFAULT 0 CHECK (penalty >= 0),
    final_mark     REAL NOT NULL CHECK (final_mark >= 0),
    feedback       TEXT NOT NULL DEFAULT '',
    marked_by      INTEGER NOT NULL REFERENCES user_account(id),
    marked_at      TEXT NOT NULL
);

CREATE TABLE audit_entry (
    id             INTEGER PRIMARY KEY,
    submission_id  INTEGER NOT NULL REFERENCES submission(id),
    actor_id       INTEGER NOT NULL REFERENCES user_account(id),
    at             TEXT NOT NULL,
    action         TEXT NOT NULL CHECK (action IN ('MarkSet','MarkChanged','Approved')),
    old_value      REAL,
    new_value      REAL,
    reason         TEXT NOT NULL DEFAULT '',
    CHECK (action <> 'MarkChanged' OR length(trim(reason)) > 0)
);

CREATE INDEX idx_course_department     ON course(department_id);
CREATE INDEX idx_enrolment_student     ON enrolment(student_id);
CREATE INDEX idx_assignment_course     ON assignment(course_id);
CREATE INDEX idx_extension_lookup      ON extension(assignment_id, student_id);
CREATE INDEX idx_submission_assignment ON submission(assignment_id);
CREATE INDEX idx_audit_submission      ON audit_entry(submission_id);

-- BR6: a submission cannot be Released unless its mark has feedback.
CREATE TRIGGER trg_release_needs_feedback
BEFORE UPDATE OF status ON submission
WHEN NEW.status = 'Released'
 AND NOT EXISTS (SELECT 1 FROM mark
                 WHERE submission_id = NEW.id AND length(trim(feedback)) > 0)
BEGIN
    SELECT RAISE(ABORT, 'BR6: feedback is required before release');
END;

-- BR4: the audit log can only grow.
CREATE TRIGGER trg_audit_no_update BEFORE UPDATE ON audit_entry
BEGIN SELECT RAISE(ABORT, 'audit_entry is append-only'); END;
CREATE TRIGGER trg_audit_no_delete BEFORE DELETE ON audit_entry
BEGIN SELECT RAISE(ABORT, 'audit_entry is append-only'); END;
```

The indexes support NFR1 and NFR2: lists by course, by assignment and by student, and the extension lookup, all use an index. The trigger gives BR6 a second guard in the database, in addition to the check in ModerationService.

5.4 How the tables map to the classes

Class (design)	Table	Difference and why
User, Administrator, HeadOfDepartment, Lecturer, Student	user_account	Inheritance becomes one table with a role column (single-table mapping). The four subclasses add almost nothing (only departmentId for HeadOfDepartment), so four tables would only add joins. UserFactory rebuilds the right subclass from the role.
Role, SubmissionStatus, AuditAction (enumerations)	role, status and action columns	Stored as text with a CHECK listing the allowed words. Lookup tables would add joins for values that the code fixes anyway.
Department	department	Direct.
Course	course	Direct.
Course and Lecturer (many to many)	course_lecturer	A many-to-many association needs its own table.
Course and Student (many to many)	enrolment	Same reason.
Assignment	assignment	TimePoint becomes ISO-8601 UTC text.
Extension	extension	Direct.
Submission	submission	The unique pair (assignment, student) enforces one submission per student.
SubmissionVersion	submission_version	The content vector becomes a BLOB (file_data) and fileSize becomes file_size with a 10 MB limit.
Mark	mark	The key is submission_id, which makes the 1 to 0..1 relationship.
AuditEntry	audit_entry	optional<double> becomes a nullable REAL. Triggers make the table append-only.
Services, repositories, IPenaltyPolicy and its implementations, IClock, Transaction, request and result structs	(none)	These are behaviour or temporary data and are not stored.

6. Design traceability

This table shows where each requirement lands in the design. Use case numbers come from the Phase 2 use case list. Figure numbers refer to the figures in this document.

6.1 Functional requirements

Requirement	Use cases	Classes and methods	Tables	Figures
FR01 four roles	UC01	User, Administrator, HeadOfDepartment, Lecturer, Student, Role, UserFactory	user_account.role	2
FR02 log in, lock-out	UC01	AuthService.login, IPasswordHasher, User.isLocked, registerFailedLogin	user_account (password_hash, failed_attempts, locked_until)	2, 3, 4
FR03 change password	UC02	AuthService.changePassword, IPasswordHasher	user_account.password_hash	3, 4
FR04 manage accounts	UC03	AdminService.createUser, setActive, UserFactory	user_account	3
FR05 assign lecturers, enrol students	UC04, UC05	AdminService.assignLecturer, enrol, ICourseRepository	course_lecturer, enrolment	3, 4, 9

Requirement	Use cases	Classes and methods	Tables	Figures
FR06 create assignment	UC06	AssignmentService.createAssignment, Assignment	assignment	2, 3
FR07 view courses, assignments and effective deadline	UC09, UC13	AssignmentService.listForStudent, ICourseRepository.listForStudent, IAssignmentRepository.findExtension	course, enrolment, assignment, extension	3, 4, 7
FR08 submit assignment	UC10	SubmissionService.submit, Submission, SubmissionVersion	submission, submission_version	2, 6, 7
FR09 mark with feedback	UC15	MarkingService.recordMark, Mark	mark	2, 3, 6
FR10 replace before the deadline	UC11	SubmissionService.submit (resubmission path)	submission_version.version_no	5, 7
FR11 late submission	UC12	SubmissionService.submit (late path), SubmissionStatus.Late	submission.status	5, 7
FR12 late penalty	UC16	IPenaltyPolicy, TenPercentPerDayPolicy, MarkingService	mark (awarded_mark, penalty, final_mark)	4, 6
FR13 extensions	UC07, UC13	AssignmentService.grantExtension, Extension	extension	2, 3, 7
FR14 moderation queue	UC18	ModerationService.queue	submission.status	3
FR15 moderate, change final mark	UC19, UC20	ModerationService.approve, changeMark	mark, audit_entry	5, 6, 8
FR16 release marks	UC21, UC22, UC23, UC24	ModerationService.release, releaseAll, Submission.advanceTo	submission.status, trigger trg_release_needs_feedback	5, 6, 8
FR17 audit log of mark changes and approvals	UC15, UC17	AuditEntry, IMarkRepository.addAudit, listAudit, ModerationService.viewAudit	audit_entry (append-only triggers)	2, 4, 8
FR18 students not yet submitted	UC08	SubmissionService.listNotSubmitted	enrolment, submission	3, 9
FR19 student views released mark	UC14	SubmissionService.viewOwn (shows the mark only when Released)	mark, submission.status	3, 5

6.2 Non-functional requirements and business rules

Requirement	How the design meets it
NFR1, NFR2 (2 seconds, 800 students)	Indexes on the lookup columns (section 5.3), one local database file, no network calls. Bulk release works through single-row updates in one loop.
NFR3, NFR9 (offline, one SQLite file)	Deployment on one host (section 2.3), and file content stored as a BLOB in the same database.
NFR4 (password hashing)	IPasswordHasher implemented by SodiumPasswordHasher (Argon2id). Only the hash is stored.
NFR6 (no partly saved work)	Transaction object (commit, or roll back in the destructor) around every write that spans several rows (Figure 7).
NFR7 (mark change and audit entry together)	One Transaction in ModerationService.changeMark and MarkingService.recordMark (Figure 8).
NFR8 (70% coverage of business logic)	Layered design with interfaces, so services run in unit tests with fake repositories, a fake clock and NoPenaltyPolicy.
BR01 late penalty	TenPercentPerDayPolicy: 10% of the maximum mark for each started 24 hours, up to 3 days, then the full maximum mark (final mark 0).
BR2 students see marks only when Released	SubmissionService.viewOwn returns the mark only for status Released.
BR3 Moderated before Released	Submission.advanceTo allows only the transitions in Figure 5. Release of a Marked submission records the approval first.
BR4 mark change log	audit_entry, written in the same transaction as the mark; triggers forbid update and delete.
BR5 late work never rejected	First submission after the deadline is saved as Late (Figure 7).

Requirement	How the design meets it
BR6 feedback before release	Check in ModerationService, and trigger trg_release_needs_feedback as a second guard.

NFR5 is not listed because it does not drive any design decision in this document. It should be checked against the SRS wording at Gate 2.

7. Design decisions to confirm before development

These choices were made so the design could be finished. Each needs a yes or a change from the stakeholders before coding starts.

1. **Where the system runs.** The application and lms.db are on one host computer. Users reach it at the host or by remote terminal. A client-server design was rejected (section 2.4).
2. **Files inside the database.** Submitted files are stored as BLOBs in lms.db, within the 10 MB limit. This keeps a submission atomic (NFR6) and keeps all data in one file (NFR9). The cost is a larger database file, so back-ups copy the whole file.
3. **Password hashing library.** libsodium (Argon2id) is assumed to be allowed as the one external library besides SQLite.
4. **Penalty policy.** TenPercentPerDayPolicy is the real rule. NoPenaltyPolicy exists for tests and as a null object, and is not offered to users.
5. **Final mark is stored.** The Head of Department can change it directly (UC20), so it is not calculated on the fly.
6. **Releasing a Marked submission counts as approval.** The approval is written to the audit log and the submission passes through Moderated (BR3).
7. **Several extensions.** If a lecturer grants a student more than one extension for an assignment, the latest one counts.
8. **NFR5** to be checked against the SRS wording (section 6.2).